

RL_01_fundamentos

April 10, 2026

1 Reinforcement Learning - Parte 1: Fundamentos

Este es el primer notebook de la serie de Reinforcement Learning.

1.1 Serie Completa

Notebook	Contenido	Prerequisitos
<code>RL_01_fundamentos.ipynb</code> (este)	Fundamentos, SARSA, Q-Learning	Ninguno
<code>RL_02_dqn.ipynb</code>	DQN, Gymnasium, Práctica	Parte 1
<code>RL_03_sb3.ipynb</code>	Stable-Baselines3, PPO, SAC	Parte 1 y 2

1.2 Índice de este Notebook

1. Introducción al Reinforcement Learning
2. Conceptos Fundamentales
3. El Framework Agente-Entorno
4. Proceso de Decisión de Markov (MDP)
5. Políticas y Funciones de Valor
6. SARSA: Aprendizaje On-Policy
7. Q-Learning: Tu Primer Algoritmo

Continúa en: `RL_02_dqn.ipynb` (DQN, Gymnasium, Práctica)

1.3 Instalación de Dependencias

Ejecuta esta celda para instalar las librerías necesarias:

```
[ ]: # Instalar dependencias (ejecutar solo si es necesario)
    # Descomenta la línea siguiente si te faltan librerías:

    pip install gymnasium numpy matplotlib torch -q

    print(" Dependencias instaladas")
```

Dependencias instaladas

```
[ ]: # Imports generales que usaremos
import numpy as np
import matplotlib.pyplot as plt
import random
from collections import defaultdict
from IPython.display import clear_output
import time

# Configuración de visualización
try:
    plt.style.use('seaborn-v0_8-whitegrid')
except:
    try:
        plt.style.use('seaborn-whitegrid')
    except:
        pass # Usar estilo por defecto

print(" Imports cargados correctamente")
```

Imports cargados correctamente

1.4 ¿Qué es el Reinforcement Learning?

El **Reinforcement Learning (RL)** o **Aprendizaje por Refuerzo** es un paradigma de Machine Learning donde un **agente** aprende a tomar decisiones interactuando con un **entorno**.

1.4.1 ¿Por qué es diferente a otros tipos de ML?

A diferencia del aprendizaje supervisado (donde tenemos las respuestas correctas) o el no supervisado (donde buscamos patrones), en RL:

- **No hay un “profesor”** que diga qué acción es correcta
- El agente debe **descubrir** qué acciones son buenas mediante **prueba y error**
- Las consecuencias de una acción pueden no verse **inmediatamente** (recompensas retrasadas)
- Las acciones afectan no solo la recompensa inmediata, sino también **situaciones futuras**

1.4.2 Aplicaciones Reales del RL

Área	Aplicación	Ejemplo Real
Videojuegos	IA que supera humanos	AlphaGo (venció al campeón mundial de Go), OpenAI Five (Dota 2)
Robótica	Control de movimiento	Robots cuadrúpedos de Boston Dynamics, brazos robóticos industriales
Vehículos	Conducción autónoma	Tesla Autopilot, Waymo
Finanzas	Trading algorítmico	Gestión automática de portfolios de inversión

Área	Aplicación	Ejemplo Real
Industria	Optimización de procesos	Google redujo 40% el consumo energético de sus data centers con RL
Recomendaciones	Sistemas adaptativos	Netflix, YouTube, Spotify adaptan sus recomendaciones
Medicina	Tratamientos personalizados	Dosificación óptima de medicamentos

1.4.3 Los Tres Paradigmas del Machine Learning

Paradigma	Datos	Objetivo	Ejemplo
Supervisado	Etiquetados (X, y)	Predecir etiquetas	Clasificar imágenes de gatos/perros
No Supervisado	Sin etiquetar (X)	Encontrar patrones	Agrupar clientes por comportamiento
Refuerzo	Experiencia (s, a, r, s')	Maximizar recompensa acumulada	Aprender a jugar ajedrez

1.4.4 Analogía Detallada: Aprender a Caminar

Imagina un bebé aprendiendo a caminar. Esta es la analogía perfecta para entender RL:

Concepto RL	En el bebé	Explicación
Agente	El bebé	El que toma decisiones y actúa
Entorno	El mundo físico	Suelo, gravedad, obstáculos, muebles
Estado	Posición y equilibrio	¿Está de pie? ¿Inclinado? ¿Cayendo?
Acciones	Movimientos	Mover pierna izq/der, inclinarse, apoyarse
Recompensa (+)	Dar un paso sin caer	Sensación de logro, avance hacia objetivo
Recompensa (-)	Caerse	Dolor, frustración
Política	Estrategia aprendida	“Si me inclino mucho, debo mover la pierna”

Punto clave: El bebé **no tiene un manual** de instrucciones. Nadie le dice “contrae el músculo X al 47%”. Aprende por **experiencia directa**: intenta algo, ve qué pasa, y ajusta su estrategia.

1.4.5 El Ciclo del RL: El Corazón del Aprendizaje

Este diagrama representa la interacción fundamental que ocurre **en cada paso de tiempo**:

Acción (a)

AGENTE

ENTORNO

Estado (s), Recompensa (r)

1.4.6 Paso a Paso del Ciclo

Paso	Descripción	Ejemplo (Robot limpiador)
1	Observar: El agente observa el estado actual	“Estoy en la cocina, hay suciedad al norte”
2	Decidir: Según su política, elige una acción	“Mi política dice: si hay suciedad cerca, ir hacia ella”
3	Actuar: Ejecuta la acción en el entorno	Moverse hacia el norte
4	Transición: El entorno cambia de estado	El robot ahora está sobre la suciedad
5	Recompensa: El entorno da feedback numérico	+10 por llegar a la suciedad
6	Aprender: El agente actualiza su conocimiento	“Moverme hacia suciedad da buena recompensa”
	Repetir hasta alcanzar el objetivo o límite	Continuar hasta limpiar toda la casa

1.4.7 ¿Por qué este ciclo es tan poderoso?

1. **Generalidad:** Este mismo ciclo funciona para jugar ajedrez, conducir coches, o controlar robots
2. **Adaptabilidad:** El agente mejora con cada iteración del ciclo
3. **Autonomía:** No necesita supervisión humana constante, aprende solo
4. **Optimización:** Busca automáticamente la mejor estrategia posible

```
[ ]: # EJEMPLO PRÁCTICO 1: Simulación Simple del Ciclo RL
# =====
# Este ejemplo muestra el bucle fundamental de RL:
# Agente observa estado → elige acción → recibe recompensa → repite

import random # Para decisiones aleatorias del agente

class EntornoSimple:
    """
    Entorno minimalista: el agente debe moverse hacia la derecha.

    Estados: posición del agente (0 a 4)
    Acciones: 0 = izquierda, 1 = derecha
    """
```

```

Meta: llegar a posición 4
"""

def __init__(self):
    # Inicializamos el agente en posición 0 (extremo izquierdo)
    self.estado = 0

def reset(self):
    """Reinicia el entorno al estado inicial."""
    self.estado = 0 # Volver al inicio
    return self.estado # Devolver el estado para que el agente lo observe

def step(self, accion):
    """
    Ejecuta una acción y devuelve el resultado.

    Args:
        accion: 0 (izquierda) o 1 (derecha)

    Returns:
        nuevo_estado: posición actualizada
        recompensa: señal de feedback (+10 al llegar, -1 por paso)
        terminado: True si llegó a la meta
    """
    # Ejecutar la acción: mover izquierda o derecha
    if accion == 0: # Izquierda
        self.estado = max(0, self.estado - 1) # No salir del límite
    ↪ izquierda

    else: # Derecha
        self.estado = min(4, self.estado + 1) # No salir del límite derecho

    # Calcular recompensa según el resultado
    if self.estado == 4: # ¡Llegó a la meta!
        recompensa = 10 # Recompensa alta por completar el objetivo
        terminado = True # El episodio termina
    else:
        recompensa = -1 # Pequeño castigo por cada paso (incentiva
    ↪ rapidez)

        terminado = False # El episodio continúa

    return self.estado, recompensa, terminado

class AgenteAleatorio:
    """
    Agente que elige acciones al azar.
    Sirve como baseline: ¿qué tan bien lo hace sin aprender?
    """

```

```

"""

def elegir_accion(self, estado):
    """
    Selecciona una acción aleatoria.

    En RL real, aquí iría la lógica de la política.
    Este agente ignora el estado y elige al azar.
    """
    # 50% probabilidad de ir a izquierda o derecha
    return random.choice([0, 1])

def ejecutar_episodio(entorno, agente, max_pasos=20):
    """
    Ejecuta un episodio completo del ciclo RL.

    Un episodio = desde el estado inicial hasta llegar a la meta
    (o hasta agotar el máximo de pasos).
    """
    # 1. RESET: Obtener estado inicial
    estado = entorno.reset()

    recompensa_total = 0 # Acumular recompensas del episodio
    historial = []       # Guardar la trayectoria para análisis

    # 2. BUCLE PRINCIPAL DEL RL
    for paso in range(max_pasos):
        # 2a. OBSERVAR: El agente ve el estado actual
        # 2b. DECIDIR: El agente elige una acción según su política
        accion = agente.elegir_accion(estado)

        # 2c. ACTUAR: El entorno ejecuta la acción
        # 2d. RECIBIR FEEDBACK: El entorno devuelve nuevo estado y recompensa
        nuevo_estado, recompensa, terminado = entorno.step(accion)

        # Guardar esta transición (s, a, r, s') para análisis
        historial.append({
            'paso': paso + 1,
            'estado': estado,
            'accion': 'izquierda' if accion == 0 else 'derecha',
            'recompensa': recompensa,
            'nuevo_estado': nuevo_estado
        })

    # Acumular recompensa total del episodio
    recompensa_total += recompensa

```

```

        # Preparar siguiente iteración
        estado = nuevo_estado

        # 2e. VERIFICAR: ¿Terminó el episodio?
        if terminado:
            break

    return recompensa_total, historial

# === EJECUTAR LA SIMULACIÓN ===
print("=" * 60)
print("SIMULACIÓN DEL CICLO RL")
print("=" * 60)

# Crear el entorno y el agente
entorno = EntornoSimple()
agente = AgenteAleatorio()

# Ejecutar un episodio y mostrar paso a paso
recompensa, historial = ejecutar_episodio(entorno, agente)

print("\nTrayectoria del agente:")
print("-" * 50)
for h in historial:
    print(f"Paso {h['paso']:2d}: Estado {h['estado']} → {h['accion']:>9s} → "
          f"Estado {h['nuevo_estado']} (R={h['recompensa']:3d})")

print("-" * 50)
print(f"Recompensa total del episodio: {recompensa}")
print(f"Número de pasos: {len(historial)}")

# Ejecutar múltiples episodios para ver la variabilidad
print("\n" + "=" * 60)
print("ESTADÍSTICAS DE 100 EPISODIOS")
print("=" * 60)

recompensas = []
for _ in range(100):
    r, _ = ejecutar_episodio(entorno, agente)
    recompensas.append(r)

print(f"Recompensa promedio: {np.mean(recompensas):.2f}")
print(f"Recompensa mínima: {min(recompensas)}")
print(f"Recompensa máxima: {max(recompensas)}")
print(f"\n Un agente que APRENDE debería mejorar estas estadísticas")

```

```
=====
SIMULACIÓN DEL CICLO RL
=====
```

Trayectoria del agente:

```
-----
Paso 1: Estado 0 → derecha → Estado 1 (R= -1)
Paso 2: Estado 1 → derecha → Estado 2 (R= -1)
Paso 3: Estado 2 → izquierda → Estado 1 (R= -1)
Paso 4: Estado 1 → izquierda → Estado 0 (R= -1)
Paso 5: Estado 0 → derecha → Estado 1 (R= -1)
Paso 6: Estado 1 → izquierda → Estado 0 (R= -1)
Paso 7: Estado 0 → izquierda → Estado 0 (R= -1)
Paso 8: Estado 0 → derecha → Estado 1 (R= -1)
Paso 9: Estado 1 → izquierda → Estado 0 (R= -1)
Paso 10: Estado 0 → derecha → Estado 1 (R= -1)
Paso 11: Estado 1 → izquierda → Estado 0 (R= -1)
Paso 12: Estado 0 → izquierda → Estado 0 (R= -1)
Paso 13: Estado 0 → izquierda → Estado 0 (R= -1)
Paso 14: Estado 0 → derecha → Estado 1 (R= -1)
Paso 15: Estado 1 → izquierda → Estado 0 (R= -1)
Paso 16: Estado 0 → derecha → Estado 1 (R= -1)
Paso 17: Estado 1 → derecha → Estado 2 (R= -1)
Paso 18: Estado 2 → derecha → Estado 3 (R= -1)
Paso 19: Estado 3 → derecha → Estado 4 (R=+10)
-----
```

Recompensa total del episodio: -8

Número de pasos: 19

```
=====
ESTADÍSTICAS DE 100 EPISODIOS
=====
```

Recompensa promedio: -6.95

Recompensa mínima: -20

Recompensa máxima: 7

Un agente que APRENDE debería mejorar estas estadísticas

1.4.8 Preguntas para Reflexionar

Después de observar al agente aleatorio:

1. **Eficiencia:** ¿Cuántos pasos tomó el agente aleatorio? ¿Cuál sería el mínimo posible?
 - Respuesta: El mínimo es **5 pasos** (siempre ir a la derecha)
2. **Recompensa:** ¿Por qué la recompensa total es negativa a pesar de obtener +10 al final?
 - Respuesta: Cada paso cuesta -1. Si tardas más de 10 pasos, pierdes más de lo que ganas.
3. **Política óptima:** ¿Cuál sería la **política perfecta** para este problema?
 - Respuesta: Siempre ir a la derecha, sin importar el estado actual.

4. **Limitación del agente aleatorio:** ¿Por qué el agente aleatorio no es bueno?

- Respuesta: No **aprende** de la experiencia. Cada decisión es independiente de las anteriores.

Punto clave: El objetivo del RL es encontrar esa política óptima **automáticamente**, sin que nosotros la programemos explícitamente.

```
[ ]: # EJEMPLO PRÁCTICO 2: Tipos de Políticas
# =====
# Una política es la "estrategia" del agente: cómo decide qué hacer.
# Comparamos tres tipos de políticas fundamentales.

class PoliticaAleatoria:
    """
    Política que elige acciones completamente al azar.

    Útil como BASELINE: ¿qué tan bien lo hace sin ninguna inteligencia?
    También es útil para exploración pura.
    """
    def __init__(self, n_acciones):
        """
        Args:
            n_acciones: Número de acciones posibles
        """
        self.n_acciones = n_acciones

    def elegir(self, estado):
        """
        Elige una acción ignorando el estado.

        Args:
            estado: Estado actual (no se usa, pero mantiene la interfaz)

        Returns:
            accion: Índice de acción aleatorio
        """
        # random.randint(a, b) incluye ambos extremos
        return random.randint(0, self.n_acciones - 1)

class PoliticaDeterminista:
    """
    Política que siempre elige la misma acción para cada estado.

    Dada una tabla estado→acción, siempre devuelve la misma acción
    para el mismo estado. No hay aleatoriedad.

    Ejemplo: "En el estado A, siempre ir a la derecha"
    """
```

```

"""
def __init__(self, tabla_acciones):
    """
    Args:
        tabla_acciones: Diccionario {estado: accion}
    """
    # La tabla mapea cada estado a su acción óptima
    self.tabla = tabla_acciones

def elegir(self, estado):
    """
    Busca en la tabla qué acción corresponde a este estado.

    Args:
        estado: Estado actual

    Returns:
        accion: Acción determinada por la tabla
    """
    # Si el estado no está en la tabla, devuelve 0 por defecto
    return self.tabla.get(estado, 0)

class PoliticaEpsilonGreedy:
    """
    Política que balancea EXPLORACIÓN vs EXPLOTACIÓN.

    - Con probabilidad (epsilon): elige al AZAR (explorar)
    - Con probabilidad 1- : elige la MEJOR acción conocida (explotar)

    = 0.0: Siempre explota (puede quedarse en óptimos locales)
    = 1.0: Siempre explora (nunca usa lo aprendido)
    = 0.1: 10% exploración, 90% explotación (típico en práctica)
    """
    def __init__(self, Q, epsilon=0.1):
        """
        Args:
            Q: Tabla Q (diccionario estado → array de Q-valores)
            epsilon: Probabilidad de explorar (0.0 a 1.0)
        """
        self.Q = Q          # Referencia a la tabla de valores
        self.epsilon = epsilon

    def elegir(self, estado):
        """
        Elige acción usando estrategia -greedy.

```

```

    Args:
        estado: Estado actual

    Returns:
        accion: Acción seleccionada
    """
    # Generar número aleatorio entre 0 y 1
    if random.random() < self.epsilon:
        # EXPLORAR: Acción aleatoria
        # Esto permite descubrir nuevas estrategias
        n_acciones = len(self.Q[estado])
        return random.randint(0, n_acciones - 1)
    else:
        # EXPLOTAR: Mejor acción según Q
        # np.argmax devuelve el índice del valor máximo
        return np.argmax(self.Q[estado])

# === DEMOSTRACIÓN DE POLÍTICAS ===
print("=" * 50)
print("COMPARACIÓN DE POLÍTICAS")
print("=" * 50)

# Crear tabla Q de ejemplo (valores inventados para demostración)
# Estado (0,0) tiene 4 acciones con diferentes Q-valores
Q_ejemplo = {
    (0, 0): np.array([1.0, 5.0, 2.0, 1.5]), # La acción 1 (→) es la mejor
    (0, 1): np.array([2.0, 3.0, 4.0, 1.0]), # La acción 2 (↓) es la mejor
    (1, 0): np.array([0.5, 0.5, 0.5, 0.5]), # Todas iguales
}

estado = (0, 0)
print(f"\nEstado: {estado}")
print(f"Q-valores: {Q_ejemplo[estado]}")
print(f"Acciones: [↑=0, →=1, ↓=2, ←=3]")
print(f"Mejor acción: → (índice 1, Q=5.0)")

# Probar política aleatoria
print("\n--- Política Aleatoria ---")
pol_aleatoria = PoliticaAleatoria(n_acciones=4)
acciones = [pol_aleatoria.elegir(estado) for _ in range(10)]
print(f"10 elecciones: {acciones}")
print("(Cada acción tiene ~25% de probabilidad)")

# Probar política epsilon-greedy con diferentes epsilon
print("\n--- Política -Greedy ---")
for eps in [0.0, 0.1, 0.5, 1.0]:

```

```

pol_eg = PoliticaEpsilonGreedy(Q_ejemplo, epsilon=eps)
acciones = [pol_eg.elegir(estado) for _ in range(20)]
cuenta_optima = acciones.count(1) # Acción 1 es la óptima
print(f"={eps}: {acciones[:10]}... (óptima {cuenta_optima}/20 veces)")

print("\n Observa cómo =0 siempre elige 1, mientras =1 es aleatorio")

```

=====

COMPARACIÓN DE POLÍTICAS

=====

Estado: (0, 0)
 Q-valores: [1. 5. 2. 1.5]
 Acciones: [↑=0, →=1, ↓=2, ←=3]
 Mejor acción: → (índice 1, Q=5.0)

--- Política Aleatoria ---

10 elecciones: [2, 1, 2, 3, 3, 2, 1, 3, 1, 0]
 (Cada acción tiene ~25% de probabilidad)

--- Política -Greedy ---

=0.0: [np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1),
 np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1)]... (óptima
 20/20 veces)
 =0.1: [np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1),
 np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1)]... (óptima
 20/20 veces)
 =0.5: [0, np.int64(1), 0, np.int64(1), np.int64(1), 2, 3, np.int64(1),
 np.int64(1), np.int64(1)]... (óptima 15/20 veces)
 =1.0: [0, 0, 2, 0, 1, 2, 3, 1, 2, 1]... (óptima 4/20 veces)

Observa cómo =0 siempre elige 1, mientras =1 es aleatorio

1.5 El Dilema Exploración vs Explotación

Este es uno de los **desafíos fundamentales** del RL y aparece en muchos problemas de la vida real.

1.5.1 El Problema

- **Exploración:** Probar acciones nuevas/desconocidas para descubrir mejores estrategias
- **Explotación:** Usar el conocimiento actual para maximizar recompensa inmediata

El dilema: Si solo explotas, nunca descubrirás opciones mejores. Si solo exploras, nunca aprovecharás lo que ya sabes.

1.5.2 Analogía: Elegir Restaurante

Imagina que estás de vacaciones 7 días en una ciudad nueva:

Estrategia	Descripción	Resultado
Solo explotar	Día 1 encuentras un restaurante “bueno” y vas los 7 días	Nunca descubres el restaurante increíble de la calle de al lado
Solo explorar	Cada día pruebas un restaurante diferente	El día 7 descubres el mejor, pero solo comes ahí 1 vez
Balance	Primeros días exploras, últimos días explotas el mejor	Maximizas la experiencia gastronómica total

1.5.3 Estrategia -greedy (Epsilon-Greedy)

La solución más simple y utilizada:

Con probabilidad ϵ → Acción ALEATORIA (explorar)

Con probabilidad $1-\epsilon$ → MEJOR acción conocida (explotar)

Ejemplo con $\epsilon = 0.1$: - 10% de las veces: Prueba algo nuevo - 90% de las veces: Hace lo que cree que es mejor

1.5.4 Estrategias de Exploración

Estrategia	Cómo funciona	Ventajas	Desventajas
-greedy	Aleatoria con prob	Simple de implementar	Exploración no inteligente
-decay	disminuye con el tiempo	Más explotación al final	Requiere ajustar el decay
UCB	Bonus por incertidumbre	Explora lo menos conocido	Más complejo
Boltzmann	Softmax sobre Q-valores	Favorece acciones prometedoras	Requiere ajustar temperatura

1.5.5 ¿Cómo elegir ?

Valor	Comportamiento	Cuándo usarlo
0.0	100% explotación	Solo evaluación (nunca entrenamiento)
0.01	Casi solo explotación	Fase final del entrenamiento
0.1	Balance estándar	La mayoría de problemas
0.3	Mucha exploración	Inicio del entrenamiento, entornos complejos
1.0	100% aleatorio	Solo para comparar con línea base

Práctica común: Empezar con ϵ alto (0.5-1.0) y reducirlo gradualmente hasta ϵ pequeño (0.01-0.1).

```
[ ]: # EJEMPLO PRÁCTICO 3: Visualizando Exploración vs Explotación

def simular_epsilon_greedy(epsilon, n_pasos=1000):
    """Simula el problema del bandido multi-brazo"""
    # 3 máquinas tragamonedas con diferentes probabilidades de premio
    probabilidades_reales = [0.2, 0.5, 0.8] # La máquina 3 es la mejor

    # Estimaciones del agente (inicialmente no sabe nada)
    estimaciones = [0.0, 0.0, 0.0]
    conteos = [0, 0, 0]

    recompensa_acumulada = 0
    historial = []
    selecciones = []

    for paso in range(n_pasos):
        # Seleccionar acción con -greedy
        if random.random() < epsilon:
            accion = random.randint(0, 2) # Explorar
        else:
            accion = np.argmax(estimaciones) # Explotar

        selecciones.append(accion)

        # Obtener recompensa (1 con probabilidad p, 0 sino)
        recompensa = 1 if random.random() < probabilidades_reales[accion] else 0
        recompensa_acumulada += recompensa

        # Actualizar estimaciones (promedio incremental)
        conteos[accion] += 1
        estimaciones[accion] += (recompensa - estimaciones[accion]) /
        conteos[accion]

        historial.append(recompensa_acumulada / (paso + 1))

    return historial, selecciones

# Comparar diferentes valores de epsilon
epsilons = [0.0, 0.1, 0.3, 0.5, 1.0]
colores = ['red', 'orange', 'green', 'blue', 'purple']

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfico 1: Recompensa promedio a lo largo del tiempo
for eps, color in zip(epsilons, colores):
    historial, _ = simular_epsilon_greedy(eps)
```

```

axes[0].plot(historial, label=f'  $\epsilon$  = {eps}', color=color, alpha=0.7)

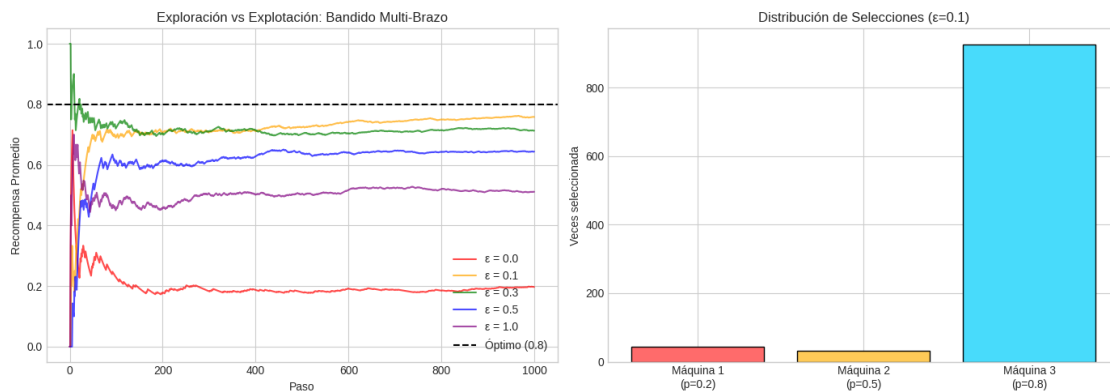
axes[0].axhline(y=0.8, color='black', linestyle='--', label='Óptimo (0.8)')
axes[0].set_xlabel('Paso')
axes[0].set_ylabel('Recompensa Promedio')
axes[0].set_title('Exploración vs Explotación: Bandido Multi-Brazo')
axes[0].legend()

# Gráfico 2: Distribución de acciones para  $\epsilon=0.1$ 
_, selecciones = simular_epsilon_greedy(0.1, 1000)
conteo_acciones = [selecciones.count(0), selecciones.count(1), selecciones.
    ↪count(2)]
colores_barras = ['#ff6b6b', '#feca57', '#48dbfb']
axes[1].bar([0, 1, 2], conteo_acciones, color=colores_barras, edgecolor='black')
axes[1].set_xticks([0, 1, 2])
axes[1].set_xticklabels(['Máquina 1\n(p=0.2)', 'Máquina 2\n(p=0.5)', 'Máquina 3\n(p=0.8)'])
axes[1].set_ylabel('Veces seleccionada')
axes[1].set_title('Distribución de Selecciones ( $\epsilon=0.1$ )')

plt.tight_layout()
plt.show()

print("\n Observaciones:")
print("-  $\epsilon=0$  (greedy puro): Se queda con la primera máquina que parece buena")
print("-  $\epsilon=0.1$ : Buen balance, encuentra la óptima y la explota")
print("-  $\epsilon=1$  (aleatorio puro): Nunca explota, recompensa promedio = 0.5)

```



Observaciones:

- $\epsilon=0$ (greedy puro): Se queda con la primera máquina que parece buena
- $\epsilon=0.1$: Buen balance, encuentra la óptima y la explota
- $\epsilon=1$ (aleatorio puro): Nunca explota, recompensa promedio = 0.5

```
[ ]: # EJEMPLO PRÁCTICO 4: Entorno de Cuadrícula (GridWorld)
# =====
# GridWorld es el entorno clásico para aprender RL.
# Simple pero contiene todos los elementos fundamentales.

class GridWorld:
    """
    Un mundo de cuadrícula simple donde el agente debe llegar a la meta.

    Mapa 4x4:
    +---+---+---+---+
    | S | . | . | . |   S = Start (inicio, posición 0,0)
    +---+---+---+---+
    | . | X | . | . |   X = Obstáculo (pared, no se puede atravesar)
    +---+---+---+---+
    | . | . | . | . |   . = Celda vacía (se puede caminar)
    +---+---+---+---+
    | . | . | . | G |   G = Goal (meta, posición 3,3)
    +---+---+---+---+

    Recompensas:
    - Llegar a G: +10 (y termina el episodio)
    - Cada paso: -1 (incentiva encontrar el camino más corto)
    """

    def __init__(self):
        """Inicializa el entorno con sus parámetros fijos."""
        # Tamaño de la cuadrícula (4x4 = 16 estados posibles)
        self.grid_size = 4

        # Posiciones especiales (fila, columna)
        self.start = (0, 0)      # Esquina superior izquierda
        self.goal = (3, 3)       # Esquina inferior derecha
        self.obstacle = (1, 1)   # Un obstáculo en el camino

        # Estado actual del agente (empieza en start)
        self.state = self.start

        # DEFINICIÓN DE ACCIONES
        # Cada acción es un desplazamiento (delta_fila, delta_columna)
        self.actions = {
            0: (-1, 0), # Arriba: fila -1, columna igual
            1: (0, 1),  # Derecha: fila igual, columna +1
            2: (1, 0),  # Abajo: fila +1, columna igual
            3: (0, -1)  # Izquierda: fila igual, columna -1
        }
    }
```



```

# Nombres para visualización
self.action_names = ['↑', '→', '↓', '←']

def reset(self):
    """
    Reinicia el entorno al estado inicial.

    Se llama al inicio de cada episodio.

    Returns:
        state: El estado inicial (posición de inicio)
    """
    self.state = self.start # Volver a la posición inicial
    return self.state       # El agente necesita saber dónde está

def step(self, action):
    """
    Ejecuta una acción y devuelve el resultado.

    Esta es la función principal del entorno. Implementa la
    dinámica del mundo: cómo responde a las acciones del agente.

    Args:
        action: Índice de la acción (0=↑, 1=→, 2=↓, 3=←)

    Returns:
        new_state: Nueva posición tras la acción
        reward: Recompensa recibida
        done: True si el episodio terminó (llegó a meta)
    """
    # PASO 1: Calcular la nueva posición tentativa
    delta = self.actions[action] # Obtener desplazamiento
    new_row = self.state[0] + delta[0] # Nueva fila
    new_col = self.state[1] + delta[1] # Nueva columna
    new_state = (new_row, new_col)

    # PASO 2: Verificar si el movimiento es válido
    # Un movimiento es válido si:
    # - No sale de los límites de la cuadrícula
    # - No choca contra el obstáculo
    dentro_limites = (0 <= new_row < self.grid_size and
                     0 <= new_col < self.grid_size)
    no_es_obstaculo = (new_state != self.obstacle)

    if dentro_limites and no_es_obstaculo:
        # Movimiento válido: actualizar posición
        self.state = new_state

```

```

        # Si no es válido, el agente se queda donde estaba

    # PASO 3: Calcular recompensa según el estado resultante
    if self.state == self.goal:
        reward = 10      # ¡Llegó a la meta! Recompensa alta
        done = True      # El episodio termina
    else:
        reward = -1      # Penalización por paso (incentiva rapidez)
        done = False     # El episodio continúa

    return self.state, reward, done

def render(self):
    """
    Visualiza el estado actual del entorno en la consola.

    Útil para debugging y para ver qué está haciendo el agente.
    """
    # Crear grid vacío
    grid = [['.' for _ in range(self.grid_size)]
            for _ in range(self.grid_size)]

    # Marcar posiciones especiales
    grid[self.obstacle[0]][self.obstacle[1]] = 'X' # Obstáculo
    grid[self.goal[0]][self.goal[1]] = 'G'          # Meta
    grid[self.state[0]][self.state[1]] = 'A'        # Agente

    # Imprimir
    print("\n" + "-" * (self.grid_size * 2 + 1))
    for row in grid:
        print("|" + " ".join(row) + "|")
    print("-" * (self.grid_size * 2 + 1))

# === DEMOSTRACIÓN DEL ENTORNO ===
print("=" * 50)
print("DEMOSTRACIÓN DE GRIDWORLD")
print("=" * 50)

# Crear instancia del entorno
env = GridWorld()

# Resetear y mostrar estado inicial
estado = env.reset()
print(f"\nEstado inicial: {estado}")
env.render()

```

```

# Ejecutar algunas acciones de ejemplo
print("\nEjecutando secuencia de acciones: → → ↓ ↓ →")
acciones = [1, 1, 2, 2, 1] # derecha, derecha, abajo, abajo, derecha

for i, accion in enumerate(acciones):
    nuevo_estado, recompensa, terminado = env.step(accion)
    print(f"\nAcción {env.action_names[accion]}: Estado {nuevo_estado}, R={recompensa}")
    env.render()
    if terminado:
        print("\n¡Meta alcanzada!")
        break

```

```

=====
DEMOSTRACIÓN DE GRIDWORLD
=====

```

Estado inicial: (0, 0)

```

-----
| A . . . |
| . X . . |
| . . . . |
| . . . G |
-----

```

Ejecutando secuencia de acciones: → → ↓ ↓ →

Acción →: Estado (0, 1), R=-1

```

-----
| . A . . |
| . X . . |
| . . . . |
| . . . G |
-----

```

Acción →: Estado (0, 2), R=-1

```

-----
| . . A . |
| . X . . |
| . . . . |
| . . . G |
-----

```

Acción ↓: Estado (1, 2), R=-1

```

-----
| . . . . |
| . X A . |
| . . . . |
| . . . G |
-----

```

Acción ↓: Estado (2, 2), R=-1

```

-----
| . . . . |
| . X . . |
| . . A . |
| . . . G |
-----

```

Acción →: Estado (2, 3), R=-1

```

-----
| . . . . |
| . X . . |
| . . . A |
| . . . G |
-----

```

```

[ ]: # EJEMPLO PRÁCTICO 5: Impacto del Factor de Descuento

def calcular_retorno(recompensas, gamma):
    """Calcula el retorno descontado  $G_t$ """
    G = 0
    for i, r in enumerate(recompensas):
        G += (gamma ** i) * r
    return G

# Ejemplo: Recompensas de un episodio
recompensas = [-1, -1, -1, -1, 10] # 4 pasos de penalización, luego recompensa

print("=" * 50)
print("IMPACTO DEL FACTOR DE DESCUENTO ( )")
print("=" * 50)
print(f"\nRecompensas del episodio: {recompensas}")
print("(4 pasos con -1, luego +10 al llegar a la meta)\n")

gammas = [0.0, 0.5, 0.9, 0.99, 1.0]
retornos = []

for gamma in gammas:

```

```

    G = calcular_retorno(recompensas, gamma)
    retornos.append(G)
    print(f" = {gamma:.2f} → Retorno G = {G:.2f}")

# Visualización
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Gráfico 1: Retorno vs Gamma
axes[0].bar(range(len(gammas)), retornos, color='steelblue')
axes[0].set_xticks(range(len(gammas)))
axes[0].set_xticklabels([f'={g}' for g in gammas])
axes[0].set_ylabel('Retorno (G)')
axes[0].set_title('Retorno según Factor de Descuento')
axes[0].axhline(y=0, color='red', linestyle='--')

# Gráfico 2: Decaimiento del valor de recompensas futuras
pasos = np.arange(0, 20)
for gamma in [0.5, 0.9, 0.99]:
    valores = gamma ** pasos
    axes[1].plot(pasos, valores, label=f' = {gamma}')

axes[1].set_xlabel('Pasos en el futuro')
axes[1].set_ylabel('Peso de la recompensa')
axes[1].set_title('Decaimiento del Valor de Recompensas Futuras')
axes[1].legend()

plt.tight_layout()
plt.show()

print("\n Interpretación:")
print("- =0: Solo ve la primera recompensa (-1). Muy miope.")
print("- =0.9: Balance entre presente y futuro.")
print("- =1: Todo el episodio vale igual (-1-1-1-1+10 = 6).")

```

```

=====
IMPACTO DEL FACTOR DE DESCUENTO ( )
=====

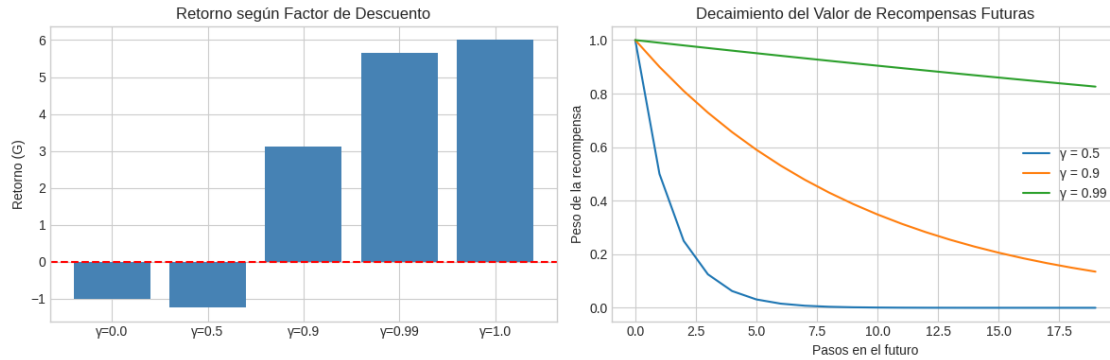
```

Recompensas del episodio: [-1, -1, -1, -1, 10]
 (4 pasos con -1, luego +10 al llegar a la meta)

```

= 0.00 → Retorno G = -1.00
= 0.50 → Retorno G = -1.25
= 0.90 → Retorno G = 3.12
= 0.99 → Retorno G = 5.67
= 1.00 → Retorno G = 6.00

```



Interpretación:

- $\gamma=0$: Solo ve la primera recompensa (-1). Muy miope.
- $\gamma=0.9$: Balance entre presente y futuro.
- $\gamma=1$: Todo el episodio vale igual ($-1-1-1-1+10 = 6$).

[]: # EJEMPLO PRÁCTICO 6: Calculando $V(s)$ por Monte Carlo

```
def estimar_valores_mc(env, politica, n_episodios=1000, gamma=0.9):
    """
    Estima  $V(s)$  usando Monte Carlo.
    Ejecuta muchos episodios y promedia los retornos observados.
    """
    # Diccionarios para acumular retornos
    retornos_totales = defaultdict(float)
    visitas = defaultdict(int)

    for _ in range(n_episodios):
        # Generar un episodio
        episodio = []
        estado = env.reset()

        for _ in range(100): # Máximo 100 pasos
            accion = politica.seleccionar_accion(estado)
            nuevo_estado, recompensa, terminado = env.step(accion)
            episodio.append((estado, accion, recompensa))
            estado = nuevo_estado
            if terminado:
                break

        # Calcular retornos para cada estado visitado (first-visit MC)
        G = 0
        estados_visitados = set()
```

```

    for t in reversed(range(len(episodio))):
        estado_t, _, recompensa_t = episodio[t]
        G = gamma * G + recompensa_t

        if estado_t not in estados_visitados:
            estados_visitados.add(estado_t)
            retornos_totales[estado_t] += G
            visitas[estado_t] += 1

    # Calcular  $V(s)$  = promedio de retornos
    V = {s: retornos_totales[s] / visitas[s] for s in retornos_totales}
    return V

# Estimar valores para GridWorld con política aleatoria
env = GridWorld()
politica_aleatoria = PoliticaAleatoria()

print("=" * 50)
print("ESTIMACIÓN DE  $V(s)$  - Monte Carlo")
print("=" * 50)
print("Política: Aleatoria |  $\gamma$  = 0.9 | 5000 episodios\n")

V = estimar_valores_mc(env, politica_aleatoria, n_episodios=5000, gamma=0.9)

# Crear matriz de valores
V_matrix = np.zeros((4, 4))
for (i, j), valor in V.items():
    V_matrix[i, j] = valor

# Visualizar
fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(V_matrix, cmap='RdYlGn')

# Añadir valores en cada celda
for i in range(4):
    for j in range(4):
        if (i, j) == (1, 1): # Obstáculo
            text = 'X'
        elif (i, j) == (3, 3): # Meta
            text = f'G\n{V_matrix[i, j]:.1f}'
        else:
            text = f'{V_matrix[i, j]:.1f}'
        ax.text(j, i, text, ha='center', va='center', fontsize=12,
            ↪fontweight='bold')

ax.set_title('V(s): Valor de cada Estado\n(Política Aleatoria)')

```

```

ax.set_xticks([])
ax.set_yticks([])
plt.colorbar(im, label='V(s)')
plt.show()

print("\n Interpretación:")
print("- Los estados cercanos a la meta tienen valores más altos (verdes)")
print("- Estados lejanos tienen valores más bajos (rojos)")
print("- El valor indica el retorno esperado siguiendo la política aleatoria")

```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[8], line 45
      43 # Estimar valores para GridWorld con política aleatoria
      44 env = GridWorld()
----> 45 politica_aleatoria = PoliticaAleatoria()
      47 print("=" * 50)
      48 print("ESTIMACIÓN DE V(s) - Monte Carlo")

TypeError: PoliticaAleatoria.__init__() missing 1 required positional argument:
↳ 'n_acciones'

```

```

[ ]: # EJEMPLO PRÁCTICO 7: Q-Learning desde cero
# =====
# Implementación completa del algoritmo Q-Learning.
# Este es el algoritmo fundamental de RL tabular.

class QLearningAgent:
    """
    Agente Q-Learning con tabla Q.

    La tabla Q almacena el valor estimado de cada par (estado, acción).
    El agente aprende actualizando estos valores basándose en la experiencia.
    """

    def __init__(self, n_acciones, alpha=0.1, gamma=0.99, epsilon=0.1):
        """
        Inicializa el agente con los hiperparámetros.

        Args:
            n_acciones: Número de acciones posibles (4 para GridWorld: ↑↓←→)
            alpha: Tasa de aprendizaje (0.1 = aprender 10% de cada experiencia)
            gamma: Factor de descuento (0.99 = el futuro importa casi tanto
↳ como el presente)
            epsilon: Probabilidad de exploración (0.1 = 10% acciones aleatorias)
        """

```



```

self.n_acciones = n_acciones
self.alpha = alpha      # : Cuánto peso dar a nueva información
self.gamma = gamma      # : Cuánto valorar recompensas futuras
self.epsilon = epsilon  # : Balance exploración/explotación

# TABLA Q: Diccionario que mapea estados a arrays de Q-valores
# Usamos defaultdict para crear automáticamente entradas nuevas
# Cada entrada: [Q(s,a0), Q(s,a1), Q(s,a2), Q(s,a3)]
# Inicializamos en 0 (optimismo neutral)
self.Q = defaultdict(lambda: np.zeros(n_acciones))

def seleccionar_accion(self, estado):
    """
    Selecciona una acción usando política -greedy.

    -greedy: Con probabilidad elige al azar (explorar),
    con probabilidad 1- elige la mejor acción conocida (explotar).

    Args:
        estado: Estado actual del entorno

    Returns:
        accion: Índice de la acción seleccionada (0-3)
    """
    # Generar número aleatorio entre 0 y 1
    if random.random() < self.epsilon:
        # EXPLORAR: Elegir acción aleatoria
        # Esto permite descubrir nuevas estrategias
        return random.randint(0, self.n_acciones - 1)
    else:
        # EXPLOTAR: Elegir la mejor acción conocida
        # np.argmax devuelve el índice del valor máximo
        return np.argmax(self.Q[estado])

def aprender(self, estado, accion, recompensa, nuevo_estado, terminado):
    """
    Actualiza la tabla Q usando la regla de Q-Learning.

    La ecuación de Q-Learning:
    
$$Q(s,a) \leftarrow Q(s,a) + [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$


    Args:
        estado: Estado donde se tomó la acción
        accion: Acción que se ejecutó
        recompensa: Recompensa recibida (r)
        nuevo_estado: Estado resultante (s')
        terminado: Si el episodio terminó
    """

```

```

    Returns:
        error_td: Diferencia entre predicción y realidad (útil para
↳debugging)
    """
    # PASO 1: Calcular el TARGET (valor "real" estimado)
    if terminado:
        # Si terminó, no hay futuro: el target es solo la recompensa
        target = recompensa
    else:
        # Target = recompensa +  $\gamma$  * mejor Q-valor futuro
        #  $\max_a Q(s', a)$ 
        mejor_q_futuro = np.max(self.Q[nuevo_estado])
        target = recompensa + self.gamma * mejor_q_futuro

    # PASO 2: Calcular el ERROR TD (Temporal Difference)
    # Error = lo que debería valer - lo que creíamos que valía
    q_actual = self.Q[estado][accion]
    error_td = target - q_actual

    # PASO 3: Actualizar Q hacia el target
    # Movemos Q un paso (proporcional a  $\alpha$ ) hacia el target
    self.Q[estado][accion] += self.alpha * error_td

    return error_td

def entrenar_qlearning(env, n_episodios=500):
    """
    Entrena un agente Q-Learning en el entorno dado.

    El proceso de entrenamiento:
    1. Para cada episodio:
        - Resetear entorno
        - Interactuar hasta terminar
        - Aprender de cada transición
    2. Reducir exploración gradualmente (epsilon decay)

    Args:
        env: Entorno de entrenamiento (debe tener reset() y step())
        n_episodios: Número de episodios de entrenamiento

    Returns:
        agente: Agente entrenado
        recompensas: Lista de recompensas por episodio (para graficar)
        pasos: Lista de pasos por episodio (para graficar)
    """

```

```

# Crear agente con hiperparámetros iniciales
# epsilon=0.2 significa 20% exploración al inicio
agente = QLearningAgent(n_acciones=4, alpha=0.1, gamma=0.9, epsilon=0.2)

# Listas para trackear el progreso
recompensas_por_episodio = []
pasos_por_episodio = []

# BUCLE DE ENTRENAMIENTO
for episodio in range(n_episodios):
    # Reiniciar el entorno al inicio de cada episodio
    estado = env.reset()
    recompensa_total = 0
    pasos = 0

    # BUCLE DE UN EPISODIO
    # Máximo 100 pasos para evitar bucles infinitos
    for _ in range(100):
        # 1. Elegir acción con política -greedy
        accion = agente.seleccionar_accion(estado)

        # 2. Ejecutar acción en el entorno
        nuevo_estado, recompensa, terminado = env.step(accion)

        # 3. APRENDER de esta experiencia
        # Esta es la línea clave: actualizar Q(s,a)
        agente.aprender(estado, accion, recompensa, nuevo_estado, terminado)

        # 4. Acumular métricas
        recompensa_total += recompensa
        pasos += 1

        # 5. Transición al nuevo estado
        estado = nuevo_estado

        # 6. Verificar si terminó el episodio
        if terminado:
            break

    # Guardar métricas del episodio
    recompensas_por_episodio.append(recompensa_total)
    pasos_por_episodio.append(pasos)

    # EPSILON DECAY: Reducir exploración gradualmente
    # Al inicio explora mucho, al final casi solo explota
    # 0.995^500 0.08, así que epsilon va de 0.2 a ~0.01
    agente.epsilon = max(0.01, agente.epsilon * 0.995)

```

```

return agente, recompensas_por_episodio, pasos_por_episodio

# === EJECUTAR ENTRENAMIENTO ===
print("=" * 50)
print("ENTRENAMIENTO Q-LEARNING")
print("=" * 50)

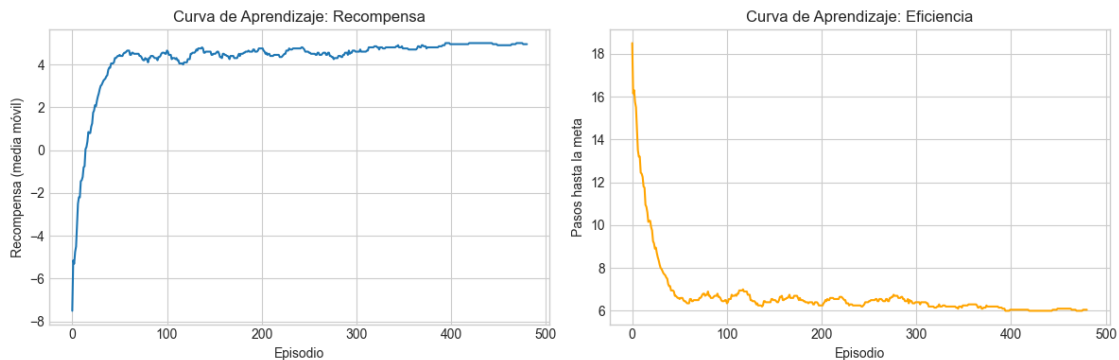
# Crear entorno GridWorld (definido anteriormente)
env = GridWorld()

# Entrenar durante 500 episodios
agente, recompensas, pasos = entrenar_qlearning(env, n_episodios=500)

# Mostrar progreso
print(f"\nProgreso del entrenamiento:")
print(f"  Primeros 50 episodios - Recompensa promedio: {np.mean(recompensas[:50]):.2f}")
print(f"  Últimos 50 episodios - Recompensa promedio: {np.mean(recompensas[-50:]):.2f}")
print(f"\n  La mejora muestra que el agente está aprendiendo!")

```

===== ENTRENAMIENTO Q-LEARNING =====



Resultados:

Recompensa promedio (últimos 50 ep): 4.94

Pasos promedio (últimos 50 ep): 6.06

```
[ ]: # EJEMPLO PRÁCTICO 8: Visualizar la política aprendida
```

```

def visualizar_politica(agente, env):
    """Muestra la política aprendida y los Q-valores"""
    flechas = ['↑', '→', '↓', '←']

    print("\n" + "=" * 50)
    print("POLÍTICA APRENDIDA")
    print("=" * 50)

    # Crear grids
    politica_grid = [[' ' for _ in range(4)] for _ in range(4)]
    valor_grid = np.zeros((4, 4))

    for i in range(4):
        for j in range(4):
            estado = (i, j)
            if estado == (1, 1): # Obstáculo
                politica_grid[i][j] = 'X'
                valor_grid[i, j] = np.nan
            elif estado == (3, 3): # Meta
                politica_grid[i][j] = 'G'
                valor_grid[i, j] = 10
            else:
                mejor_accion = np.argmax(agente.Q[estado])
                politica_grid[i][j] = flechas[mejor_accion]
                valor_grid[i, j] = np.max(agente.Q[estado])

    # Mostrar política como texto
    print("\nPolítica (flechas indican mejor acción):")
    print("-" * 17)
    for row in politica_grid:
        print("| " + " | ".join(row) + " |")
    print("-" * 17)

    # Visualizar Q-valores máximos
    fig, ax = plt.subplots(figsize=(8, 6))

    # Crear mapa de calor
    masked_grid = np.ma.masked_invalid(valor_grid)
    im = ax.imshow(masked_grid, cmap='RdYlGn')

    # Añadir flechas y valores
    for i in range(4):
        for j in range(4):
            if politica_grid[i][j] == 'X':
                ax.text(j, i, 'X', ha='center', va='center', fontsize=20,
↪fontweight='bold')
            elif politica_grid[i][j] == 'G':

```

```

        ax.text(j, i, ' ', ha='center', va='center', fontsize=20)
    else:
        ax.text(j, i, politica_grid[i][j], ha='center', va='center',
                fontsize=24, fontweight='bold')
        ax.text(j, i + 0.35, f'{valor_grid[i,j]:.1f}', ha='center',
                va='center', fontsize=10, color='black')

    ax.set_title('Política Óptima Aprendida\n(Flechas = acciones, Números = max_Q(s,a)')
    ax.set_xticks([])
    ax.set_yticks([])
    plt.colorbar(im, label='max Q(s,a)')
    plt.show()

# Visualizar
visualizar_politica(agente, env)

# Ejecutar la política aprendida
print("\n" + "=" * 50)
print("EJECUCIÓN DE LA POLÍTICA APRENDIDA")
print("=" * 50)

env = GridWorld()
estado = env.reset()
env.render()

for paso in range(10):
    accion = np.argmax(agente.Q[estado]) # Greedy (sin exploración)
    nuevo_estado, recompensa, terminado = env.step(accion)

    print(f"\nPaso {paso + 1}: {env.action_names[accion]}")
    env.render()

    estado = nuevo_estado
    if terminado:
        print("\n ¡Meta alcanzada de forma óptima!")
        break

```

```

=====
POLÍTICA APRENDIDA
=====

```

Política (flechas indican mejor acción):

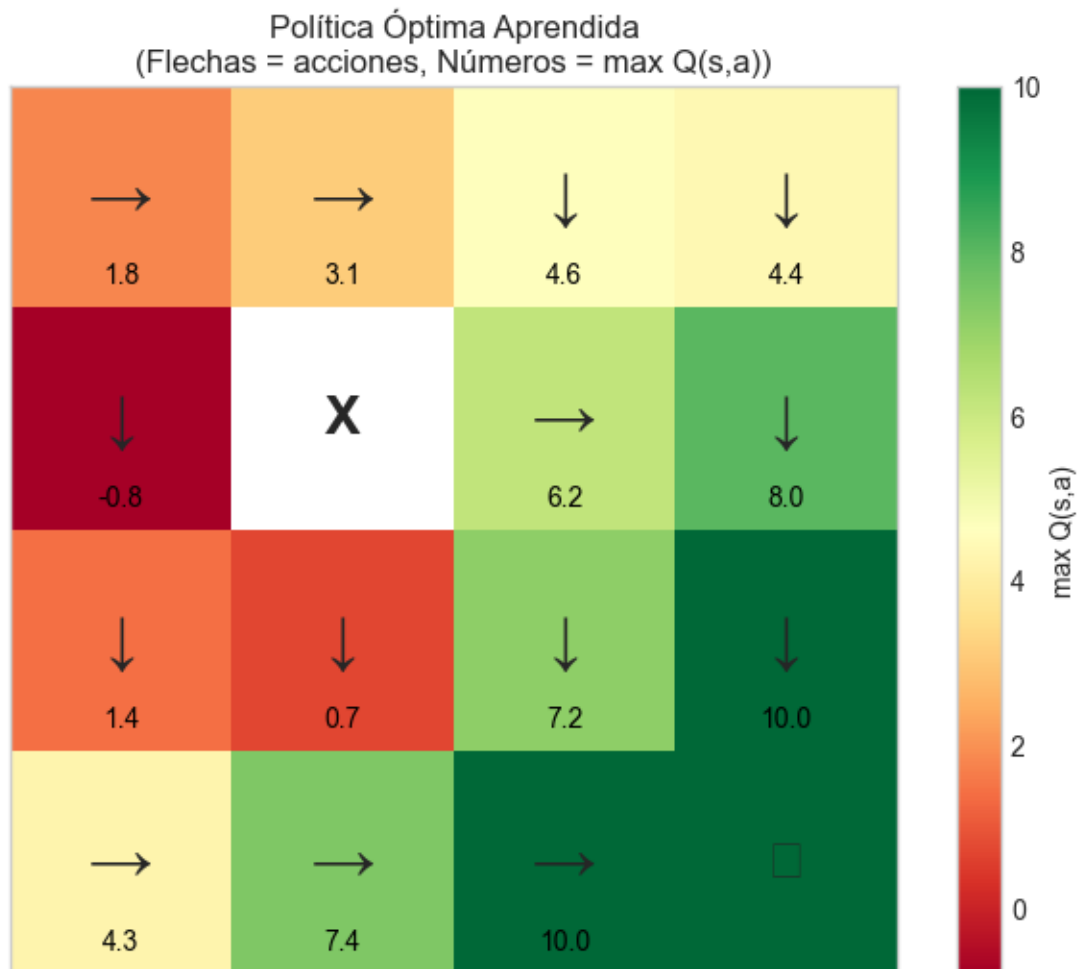
```

-----
| → | → | ↓ | ↓ |
| ↓ | X | → | ↓ |

```

```
| ↓ | ↓ | ↓ | ↓ |
| → | → | → | G |
-----
```

C:\Users\XaviLara\AppData\Local\Programs\Python\Python312\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 11088 (\N{WHITE MEDIUM STAR}) missing from font(s) Arial.
 fig.canvas.print_figure(bytes_io, **kw)



```
=====
EJECUCIÓN DE LA POLÍTICA APRENDIDA
=====
```

```
-----
| A . . . |
| . X . . |
```

	.	.	.	.	
	.	.	.	G	

Paso 1: →

	.	A	.	.	
	.	X	.	.	
	.	.	.	.	
	.	.	.	G	

Paso 2: →

	.	.	A	.	
	.	X	.	.	
	.	.	.	.	
	.	.	.	G	

Paso 3: ↓

	.	.	.	.	
	.	X	A	.	
	.	.	.	.	
	.	.	.	G	

Paso 4: →

	.	.	.	.	
	.	X	.	A	
	.	.	.	.	
	.	.	.	G	

Paso 5: ↓

	.	.	.	.	
	.	X	.	.	
	.	.	.	A	
	.	.	.	G	

Paso 6: ↓

```
-----  
| . . . . |  
| . X . . |  
| . . . . |  
| . . . A |  
-----
```

¡Meta alcanzada de forma óptima!